UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Putri Angel Li 2428240091
Kelas	SI5B
Judul project / topik API	Telekomunikasi: Data
Tautan repositori	https://github.com/Putriangelli75/tugas1-restful-2428240091
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Rabu, 21 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

Jelaskan dalam dua sampai tiga kalimat: topik API Anda, entitas utamanya, dan teknologi yang dipakai (misalnya Express.js dengan struktur Route-Controller-Model).

API yang saya buat bertema Telekomunikasi: Paket Data dengan entitas utama data-plans yang digunakan untuk mengelola informasi paket data, seperti nama paket, harga, kuota, dan masa aktif. API ini dikembangkan menggunakan Node.js dan Express.js dengan struktur Route-Controller-Model agar pengelolaan data lebih terorganisir, serta menggunakan format JSON untuk menerima dan mengirimkan data.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	{{base_url}}/data-plans		200	Menampilkan seluruh data paket data.
2	POST	{{base_url}}/data-plans	Ya	201	Menambahkan data paket data baru.
3	GET	{{base_url}}/data-plans/{{dataplansId}}		200	Menampilkan detail paket data berdasarkan ID.
4	PUT	{{base_url}}/data-plans/{{dataplansId}}	Ya	200	Memperbarui data paket data berdasarkan ID.
5	DELETE	{{base_url}}/data-plans/{{dataplansId}}	Ya	204	Menghapus data paket data berdasarkan ID.
6	GET	{{base_url}}/data-plans/{{dataplansId}}		404	Menampilkan pesan bahwa data paket data tidak ditemukan jika ID tidak tersedia.
7	POST	{{base_url}}/data-plans		401	Menolak permintaan karena API key tidak ada atau tidak valid.
8	POST	{{base_url}}/data-plans	Ya	400	Menolak permintaan karena terdapat data atau field wajib yang tidak valid atau belum diisi.
9	POST	{{base_url}}/data-plans	Ya	400	Menolak permintaan karena format JSON yang dikirim tidak valid.

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

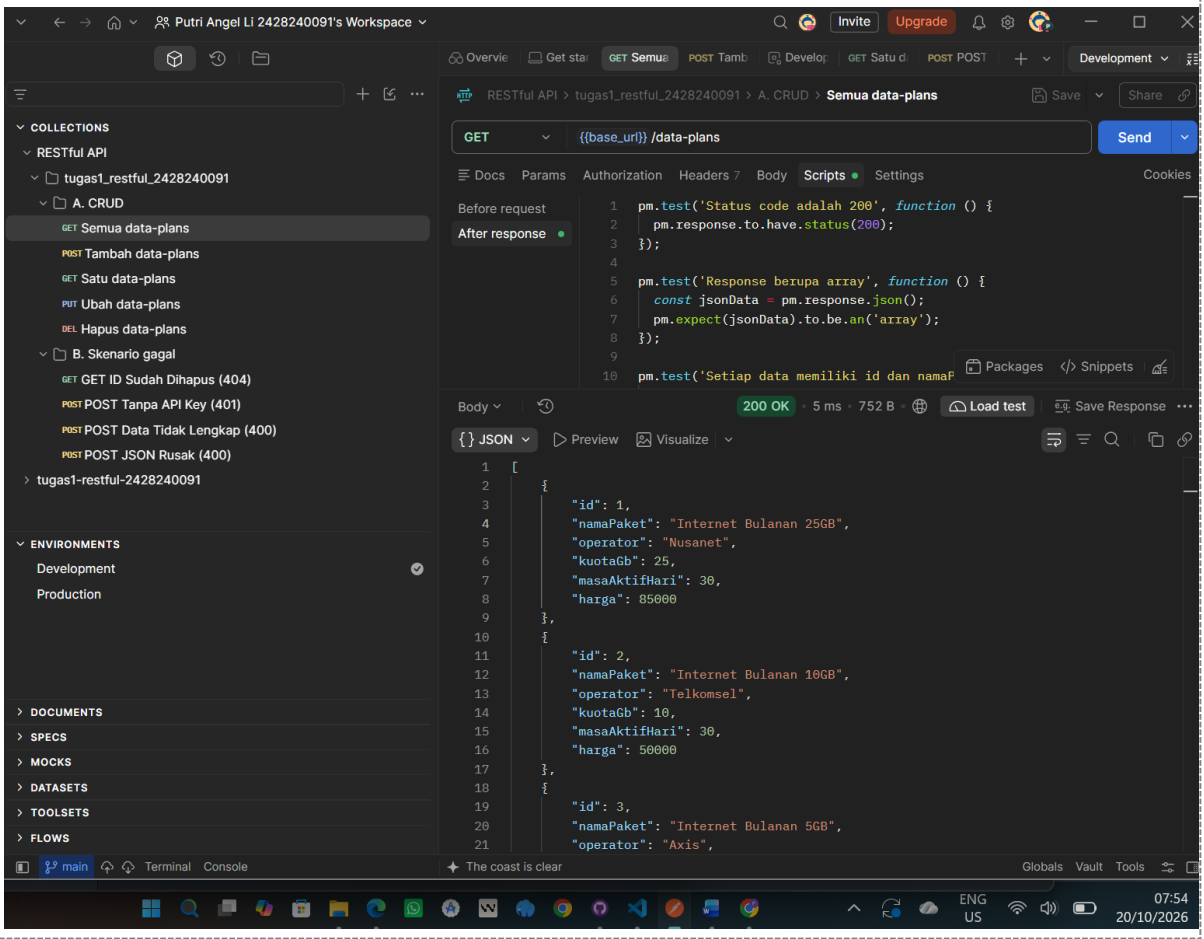
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	rahasia-tugas-paw2	Nilai header kunci API
3	dataplansId	(kosong, diisi script)	ID dari respons POST
4	base_url	tugas1-restful-2428240091.vercel.app	Menyimpan alamat server API yang sudah di-deploy ke Vercel agar request dapat dijalankan secara online.

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Jelaskan cara Anda mengelompokkan request (misalnya folder CRUD dan folder skenario gagal) dan mengapa urutannya seperti itu.

Saya mengelompokkan request pada Postman menjadi dua folder utama, yaitu A. CRUD dan B. Skenario gagal, agar pengujian API lebih terstruktur dan mudah dipahami. Folder A. CRUD berisi request GET untuk menampilkan seluruh data dan detail data, POST untuk menambahkan data, PUT untuk memperbarui data, serta DELETE untuk menghapus data paket. Sementara itu, folder B. Skenario gagal berisi pengujian kesalahan, seperti GET dengan ID yang sudah dihapus (404), POST tanpa API key (401), POST dengan data tidak lengkap (400), dan POST dengan format JSON yang salah (400).

Urutannya dimulai dari pengujian CRUD untuk memastikan fungsi utama API berjalan dengan baik, kemudian dilanjutkan dengan skenario gagal untuk memastikan API dapat menangani kesalahan dengan benar. Dengan pengelompokan ini, proses pengujian menjadi lebih rapi, sistematis, dan memudahkan pemeriksaan hasil pengujian berdasarkan status HTTP yang diharapkan.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Semua data-plans	GET	{{base_url}}/data-plans	A. CRUD
2	Tambah data-plans	POST	{{base_url}}/data-plans	A. CRUD
3	Satu data-plans	GET	{{base_url}}/data-plans/{{dataplansId}}	A. CRUD
4	Ubah data-plans	PUT	{{base_url}}/data-plans/{{dataplansId}}	A. CRUD
5	Hapus data-plans	DELETE	{{base_url}}/data-plans/{{dataplansId}}	A. CRUD
6	GET ID Sudah Dihapus (404)	GET	{{base_url}}/data-plans/{{dataplansId}}	B. Skenario gagal
7	POST Tanpa API Key (401)	POST	{{base_url}}/data-plans	B. Skenario gagal
8	POST Data Tidak Lengkap (400)	POST	{{base_url}}/data-plans	B. Skenario gagal
9	POST JSON Rusak (400)	POST	{{base_url}}/data-plans	B. Skenario gagal
10				

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Semua data-plans	pm.response.to.have.status(200)	*pm.test('Response berupa array', function () {* *const jsonData = pm.response.json();* *pm.expect(jsonData).to.be.an('array');* *});* *pm.test('Setiap data memiliki id dan namaPaket', function () {* *const jsonData = pm.response.json();* *jsonData.forEach((item) => {* *pm.expect(item).to.have.property('id');* *pm.expect(item).to.have.property('namaPaket');* *});* *});* *pm.test('Response time di bawah 500ms', function () {* *pm.expect(pm.response.responseTime).to.be.below(500);* *});* *pm.test('Data namaPaket pertama bernama "Internet Bulanan 25GB"', function () {* *const jsonData = pm.response.json();* *pm.expect(jsonData[0].namaPaket).to.eql('Internet Bulanan 25GB');* *});*

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

2	Tambah data-plans	pm.response.to.have.status(201)	pm.test('Data baru memiliki id dan namaPaket sesuai kiriman', *function* () {
			const jsonData = pm.response.json();
			pm.expect(jsonData).to.have.property('id');
			pm.expect(jsonData.namaPaket).to.eql('Internet Bulanan 50GB');
			});
			
			if (pm.response.code === 201) {
			pm.environment.set('dataplansId', pm.response.json().id);
			}

3	Satu data-plans	pm.response.to.have.status(200)	pm.test('Data yang diambil adalah Internet Bulanan 50GB', *function* () {
			const jsonData = pm.response.json();
			pm.expect(jsonData.id).to.eql(*Number*(pm.environment.get('dataplansId')));
			pm.expect(jsonData.namaPaket).to.eql('Internet Bulanan 50GB');
			});

4	Ubah data-plans	pm.response.to.have.status(200)	pm.test('Jurusan berubah, nama tetap', *function* () {
			const jsonData = pm.response.json();
			pm.expect(jsonData.operator).to.eql('Telkomsel');
			pm.expect(jsonData.namaPaket).to.eql('Internet Bulanan 50GB');
			});

5	Hapus data-plans	pm.response.to.have.status(204)	pm.test('Response tidak memiliki isi', *function* () {
			pm.expect(pm.response.text()).to.eql('');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

6	GET ID Sudah Dihapus (404)	pm.response.to.have.status(404)	pm.test('Pesan error: Data tidak ditemukan', *function* () {
			pm.expect(pm.response.json().message).to.eql('Data tidak ditemukan');
			});

7	POST Tanpa API Key (401)	pm.response.to.have.status(401)	pm.test('Pesan error: API key tidak valid', *function* () {
			pm.expect(pm.response.json().message).to.eql('API key tidak valid');
			});

8	POST Data Tidak Lengkap (400)	pm.response.to.have.status(400)	pm.test('Pesan error menyebut field wajib', *function* () {
			pm.expect(pm.response.json().message).to.include('wajib diisi');
			});

9	POST JSON Rusak (400)	pm.response.to.have.status(400)	pm.test('Respons tetap berupa JSON, bukan HTML', *function* () {
			pm.response.to.be.json;
			pm.expect(pm.response.json().message).to.eql('Format JSON tidak valid');
			});

10			

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: [nama request]
// Tempel kode test script Anda (teks, bukan gambar), lalu hapus baris ini.
GET: Satu data-plans

Kode test script :
pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Data yang diambil adalah Internet Bulanan 50GB', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.id).to.eql(Number(pm.environment.get('dataplansId')));
  pm.expect(jsonData.namaPaket).to.eql('Internet Bulanan 50GB');
});
```

Penjelasan:

Jelaskan fungsi setiap `pm.test`: apa yang diperiksa dan mengapa pemeriksaan itu penting.

Pengujian ini memeriksa apakah API mengembalikan status code 200 (OK). Status tersebut menunjukkan bahwa permintaan untuk mengambil data paket berhasil diproses. Pemeriksaan ini penting untuk memastikan endpoint GET dapat berjalan dengan baik.

pm.test('Data yang diambil adalah Internet Bulanan 50GB', function () {...}) Pengujian ini memeriksa apakah data JSON yang diterima memiliki nilai id yang sama dengan variabel environment dataplansId dan namaPaket bernilai Internet Bulanan 50GB. Pemeriksaan ini penting untuk memastikan bahwa API mengembalikan data paket yang benar sesuai ID yang diminta, bukan data paket lain.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: [nama request]
// Tempel kode test script Anda (teks, bukan gambar), lalu hapus baris ini.
```

POST : Tambah data-plans

Kode test script :

```
pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data baru memiliki id dan namaPaket sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.namaPaket).to.eql('Internet Bulanan 50GB');
});

if (pm.response.code === 201) {
  pm.environment.set('dataplansId', pm.response.json().id);
}
```

Penjelasan:

Jelaskan bagaimana data dari respons disimpan, request mana yang memakainya, dan apa yang terjadi bila urutan request ditukar.

Data dari respons POST disimpan menggunakan kode pm.environment.set('dataplansId', pm.response.json().id) jika status respons adalah 201 (Created). Kode tersebut mengambil nilai id dari data paket yang berhasil ditambahkan, kemudian menyimpannya ke variabel environment bernama dataplansId.

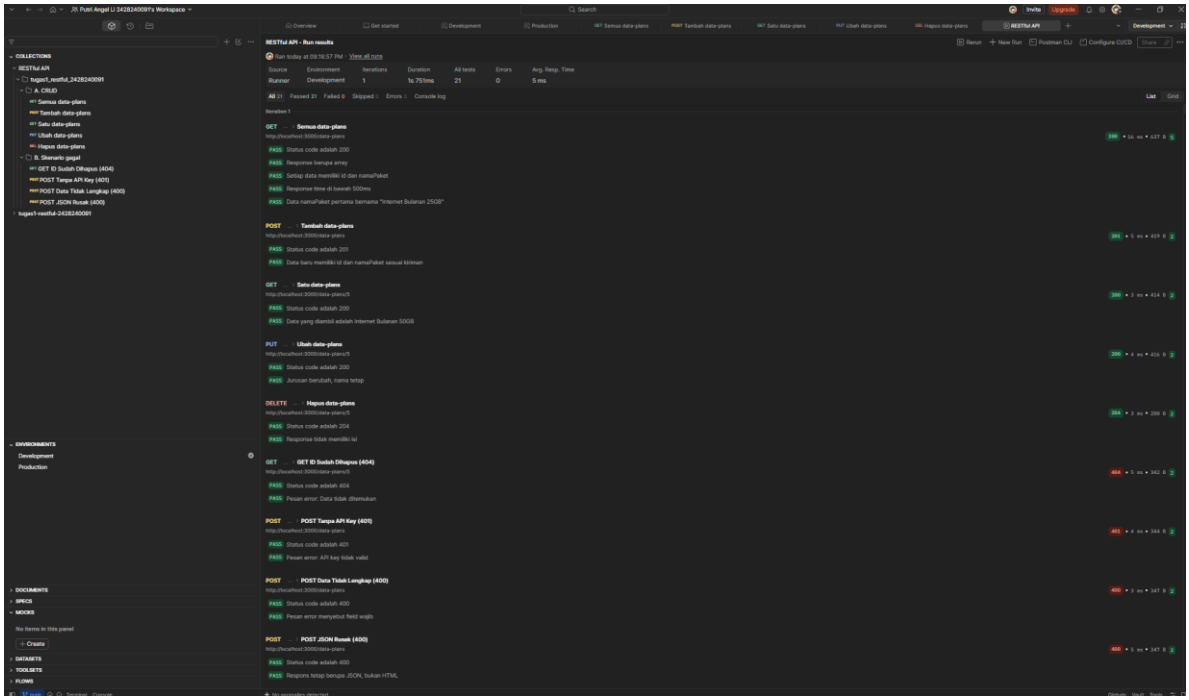
Variabel tersebut digunakan oleh request GET Satu data-plans, PUT Ubah data-plans, dan DELETE Hapus data-plans untuk mengakses data berdasarkan ID yang tersimpan. Jika urutan request ditukar sehingga GET, PUT, atau DELETE dijalankan sebelum POST, variabel dataplansId mungkin belum tersedia atau masih menyimpan ID lama. Akibatnya, request dapat gagal atau mengakses data yang tidak sesuai. Oleh karena itu, request POST sebaiknya dijalankan terlebih dahulu agar ID data tersimpan dan dapat digunakan oleh request berikutnya.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Jelaskan hasilnya: berapa request dan test yang dijalankan, apakah semuanya lulus, dan apa yang Anda lakukan sebelum menjalankan Runner (misalnya menyalakan ulang server).

Berdasarkan hasil pengujian menggunakan Postman Runner, terdapat 9 request dengan total 21 test yang dijalankan dalam 1 iterasi. Hasilnya, seluruh 21 test berhasil lulus (Passed), dengan 0 test gagal (Failed), 0 test dilewati (Skipped), dan 0 error. Pengujian mencakup fungsi CRUD serta skenario gagal, seperti ID tidak ditemukan (404), API key tidak valid (401), data tidak lengkap (400), dan format JSON rusak (400).

Sebelum menjalankan Runner, saya memastikan server Express.js berjalan dengan baik, memeriksa konfigurasi environment dan variabel base_url, serta memastikan request dan test script pada setiap endpoint sudah benar. Setelah itu, saya menjalankan seluruh koleksi melalui Runner untuk memastikan setiap endpoint berfungsi sesuai harapan dan dapat menangani kondisi berhasil maupun gagal dengan benar.

4.2 Rekap angka hasil

Nama Collection	RESTful API
Environment yang dipakai	Development
Jumlah request dijalankan	9

Jumlah test lulus	21
Jumlah test gagal	0
Tanggal dan jam pengujian	20/10/2026 jam : 09:18:57 pm

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Semua data-plans	Tidak ada kegagalan		
2	Tambah data-plans	Tidak ada kegagalan		
3	Satu data-plans	Tidak ada kegagalan		
4	Ubah data-plans	Tidak ada kegagalan		
5	Hapus data-plans	Tidak ada kegagalan		
6	GET ID Sudah Dihapus (404)	Tidak ada kegagalan		
7	POST Tanpa API Key (401)	Tidak ada kegagalan		
8	POST Data Tidak Lengkap (400)	Tidak ada kegagalan		
9	POST JSON Rusak (400)	Tidak ada kegagalan		

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

“Tidak dikerjakan”

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/Putriangelli75/tugas1-restful-2428240091
Lokasi berkas Collection (.json)	D:\tugas1-restful-2428240091\postman\collections\RESTful API
Lokasi berkas Environment (.json)	D:\tugas1-restful-2428240091\postman\environments
Berkas Environment tidak memuat kunci rahasia sungguhan	[✓] Ya [ ] Tidak
Repositori dapat dibuka dosen	[✓] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Tulis jawaban Anda di sini.

Yang paling berguna ada pada pm.response karena dari pm.response kita bisa megetahui bagian mana yang error dan perlu diperbaiki.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Tulis jawaban Anda di sini.

Perbedaannya jika manual harus di uji coba 1 per 1 sedangkan jika memakai collection runner dapat melakukan uji coba dengan 1 kali uji. Bagian yg paling sulit saat menulis test script ada dibagian pm.expect yg dimana harus mengubah nama data yang diinginkan. Cara mengatasinya tinggal mengubah saja sesuai dengan data yang kita inginkan kemudian coba jalankan apakah sesuai dengan hasil test script yg diinginkan jika tidak maka ubah kembali bagian yang salah.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai {{base_url}}, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen